Redirects are handled behind the scenes by web browsers. Normally, the only way you'll know that you've been redirected is a slight delay and the fact that the URL of the page you're viewing will have changed from the one you requested. This last point is important—as far as the browser is concerned, a redirect from a server acts pretty much the same as having an end user enter the new destination URL manually.

Redirects turn out to be important when writing well-behaved web applications. Let's look at a basic blogging application that supports comment posting. After a user has posted a comment, our application should redisplay the article, presumably with the new comment at the end.

It's tempting to code this using logic such as the following:

```
class BlogController
  def display
    @article = Article.find(params[:id])
  end

  def add_comment
    @article = Article.find(params[:id])
    comment  = Comment.new(params[:comment])
    @article.comments << comment
    if @article.save
      flash[:note] = "Thank you for your valuable comment"
    else
      flash[:note] = "We threw your worthless comment away"
    end
    # DON'T DO THIS
    render(action: 'display')
  end
end
```

The intent here was clearly to display the article after a comment has been posted. To do this, the developer ended the `add_comment` method with a call to `render(action:'display')`. This renders the `display` view, showing the updated article to the end user. But think of this from the browser's point of view. It sends a URL ending in `blog/add_comment` and gets back an index listing. As far as the browser is concerned, the current URL is still the one that ends in `blog/add_comment`. This